

Proyecto Final – EIF-201 Programación I C++ (Clion)
Docentes: Cynthia Bolaños Espinoza / Jonathan Morales Murillo
Modalidad: INDIVIDUAL
Valor Total: 20%

Sistema de Gestión Universitaria

Una universidad requiere un sistema básico de gestión académica, desarrollado en C++, que permita administrar información de personas, cursos y horarios, aplicando Programación Orientada a Objetos, manejo de memoria dinámica, estructuras de datos propias y persistencia en archivos de texto.

Este proyecto tiene como objetivo integrar los contenidos del curso EIF-201 (Programación I) y evaluar no solo que el programa funcione, sino que esté correctamente diseñado, estructurado y validado, siguiendo buenas prácticas de programación.

Este proyecto evalúa tanto el funcionamiento del sistema como la calidad del diseño y del código. Un programa que funcione, pero esté mal diseñado no obtendrá la nota máxima.

El sistema simula la forma en que una universidad administra su información académica básica, permitiendo gestionar estudiantes, profesores, cursos, aulas y horarios, manteniendo coherencia entre los datos y respetando las reglas de funcionamiento establecidas.

Este no es un programa pequeño ni un ejercicio aislado. Se trata de un ****proyecto integrador****, en el que se espera que el estudiante aplique de manera conjunta los conocimientos adquiridos durante el curso.

El énfasis no está únicamente en que el programa funcione, sino en que esté ****bien estructurado****, sea ****legible****, ****mantenible**** y refleje un ****buen diseño orientado a objetos****.

El proyecto debe resolverse utilizando únicamente los recursos propios del lenguaje C++, tales como clases, herencia, punteros, memoria dinámica, arreglos, listas enlazadas y archivos, sin utilizar librerías externas ni estructuras avanzadas no vistas en clase.

2. Objetivos de aprendizaje

Al finalizar el proyecto, el estudiante deberá demostrar que es capaz de:

- Aplicar correctamente el paradigma orientado a objetos en C++.
- Diseñar e implementar clases con responsabilidades claras.
- Utilizar herencia y polimorfismo de forma correcta.
- Implementar estructuras de datos dinámicas propias (sin STL vector, list, map, set).
- Gestionar correctamente la memoria dinámica (uso de new y delete).
- Persistir información en archivos de texto, utilizando getline y stringstream.
- Validar entradas y manejar casos bordes, sin uso de excepciones.

3. Alcance del sistema

El sistema deberá permitir gestionar las siguientes entidades:

- Persona (clase abstracta)
 - Estudiante
 - Profesor
- Curso
- Horario

Nota: Todas las entidades deberán modelarse mediante clases, siguiendo principios de diseño orientado a objetos.

4. Requerimientos funcionales

4.1 Gestión de personas

- Registrar estudiantes y profesores.
- Cada persona debe poseer al menos:
 - o Identificador único (carné o cédula)
 - o Nombre completo
- No se permite registrar personas duplicadas.

4.2 Gestión de cursos

- Registrar cursos con:
 - o Código
 - o Nombre
 - o Profesor asignado
- Un curso no puede existir sin un profesor asignado.

4.3 Gestión de horarios

- Implementar un horario semanal usando una matriz dinámica.
- El horario debe manejar:
 - o Días (filas)
 - o Franjas horarias (columnas)
- Validar conflictos de horario (no se permite asignar dos cursos en la misma franja).

5. Requerimientos técnicos obligatorios

5.1 Programación Orientada a Objetos

- Uso obligatorio de:
 - o Encapsulamiento
 - o Herencia
 - o Polimorfismo
- La clase Persona debe ser abstracta.
- Debe existir al menos un método virtual redefinido en clases derivadas.

5.2 Relaciones entre clases (UML)

El diseño debe evidenciar e implementar:

- Asociación
- Agregación
- Composición
- Herencia

El estudiante deberá entregar un diagrama UML coherente con la implementación.

5.3 Estructuras de datos (sin STL)

- Está prohibido el uso de vector, list, map, set o cualquier contenedor STL.
- Se debe implementar una lista simplemente enlazada propia para:
 - o Estudiantes
 - o Profesores
 - o Cursos
- Operaciones mínimas requeridas:
 - o Insertar
 - o Buscar
 - o Eliminar
 - o Recorrer

5.4 Memoria dinámica

- Uso obligatorio de memoria dinámica.
- Toda memoria reservada debe ser correctamente liberada.
- No deben existir fugas de memoria.

5.5 Persistencia en archivos de texto

- La información del sistema debe guardarse y cargarse desde archivos de texto.
- Uso obligatorio de:
 - o getline
 - o stringstream
- El formato del archivo debe ser documentado por el estudiante.
- El sistema debe validar líneas incompletas o corruptas.

5.6 Validaciones

- No se permite el uso de excepciones (try/catch).
 - Las validaciones deben realizarse mediante:
 - o Retornos booleanos
 - o Punteros nulos
 - o Mensajes de error controlados
-

6. Reglas de diseño (obligatorias)

El proyecto será evaluado fuertemente en diseño:

- Principio de Responsabilidad Única (SRP)
 - o Cada clase debe tener una única responsabilidad clara.
 - Principio Abierto/Cerrado (OCP)
 - o El sistema debe poder extenderse sin modificar código existente.
 - Ley de Deméter
 - o Evitar cadenas largas de llamadas a métodos.
 - Bajo acoplamiento y alta cohesión
 - o main.cpp sólo debe interactuar con una clase principal del sistema.
-

7. Casos borde obligatorios a validar

El sistema debe manejar correctamente los siguientes escenarios:

- Inserción en listas vacías.
 - Eliminación del primer, último y nodo intermedio.
 - Búsqueda de elementos inexistentes.
 - Intento insertar elementos duplicados.
 - Conflictos de horario.
 - Índices fuera de rango en la matriz dinámica.
 - Archivos inexistentes o con datos incompletos.
-

8. Restricciones

- Lenguaje: C++

- IDE recomendado: CLion
 - Prohibido:
 - Uso de STL containers
 - Uso de excepciones
 - Uso de templates
 - Uso de archivos binarios
-

9. Entregables

1. Código fuente completo del proyecto.
 2. Diagrama UML (imagen o PDF).
 3. Archivos de texto de prueba.
 4. Documento corto (1 página) describiendo:
 - Diseño general
 - Formato de archivos
 - Decisiones importantes
-

10. Criterios de evaluación

- Diseño orientado a objetos y UML: 30%
 - Estructuras dinámicas y memoria: 30%
 - Polimorfismo y herencia: 15%
 - Persistencia en archivos de texto: 15%
 - Validaciones y casos borde: 10%
-

11. Modalidad de trabajo

- Modalidad: Individual (obligatoria).
- Cada estudiante debe desarrollar su propia solución completa.
- No se permiten proyectos compartidos ni reutilización de código entre estudiantes.

- Se podrá solicitar defensa oral corta para validar autoría y comprensión del código.

12. Rúbrica de evaluación detallada

Criterio	Descripción	Puntaje
Diseño POO y UML	Clases bien definidas, relaciones correctas, UML coherente con implementación	30 pts
Estructuras dinámicas	Listas enlazadas propias funcionales, operaciones completas	20 pts
Matriz dinámica (Horario)	Uso correcto de memoria dinámica, validaciones y conflictos	10 pts
Memoria dinámica	Uso correcto de new/delete, sin fugas evidentes	10 pts
Herencia y polimorfismo	Uso real de métodos virtuales y abstracción	15 pts
Persistencia en archivos de texto	Guardar/cargar con getline y stringstream, validaciones	10 pts
Validaciones y casos borde	Manejo correcto de errores sin excepciones	5 pts
Total		100 pts

13. Casos de prueba oficiales (obligatorios)

El proyecto será evaluado utilizando, como mínimo, los siguientes casos:

Listas enlazadas

- Insertar en lista vacía
- Insertar duplicado (debe rechazarse)
- Eliminar primer nodo
- Eliminar último nodo
- Eliminar nodo intermedio

- Buscar elemento inexistente

Matriz de horario

- Asignar curso en franja válida
- Intentar asignar dos cursos en la misma franja (conflicto)
- Acceder a índice fuera de rango

Persistencia

- Guardar información con datos válidos
- Cargar archivo inexistente
- Cargar archivo con líneas incompletas

General

- Ejecutar el sistema sin datos iniciales
- Cerrar el sistema liberando toda la memoria

14. Bonus (opcional – puntos extra)

- Implementar correctamente la regla de tres en clases con memoria dinámica (+5 pts).
- Agregar un nuevo tipo de Persona sin modificar código existente (+5 pts).
- Presentar un documento de pruebas manuales ejecutadas (+5 pts).

“El proyecto integra y aplica los contenidos descritos en la Carta al Estudiante EIF-201, así como buenas prácticas de diseño orientado a objetos vistas en clase.”